Case Study: Sports Tournament Ranking System

1. Problem Statement

A sports tournament has several teams. This program stores team results, calculates total points and a tie-break value, and displays the teams in ranking order. The team with more points gets the higher rank. If two teams have equal points, goal difference is used as the tie-break value.

2. Objectives

• Store team names and match details.

• Calculate points for every team.

• Calculate goal difference as the tie-break value.

• Display the teams in ranking order.

3. Assumptions

• Win = 2 points, Draw = 1 point, Loss = 0 points.

• Goal Difference = Goals Scored - Goals Conceded.

• Higher total points gives a better rank.

• If points are equal, higher goal difference gives a better rank.

4. Algorithm

• Start.

• Enter number of teams.

• Enter team name, wins, draws, goals scored and goals conceded.

• Calculate points = (wins x 2) + draws.

• Calculate goal difference = goals scored - goals conceded.

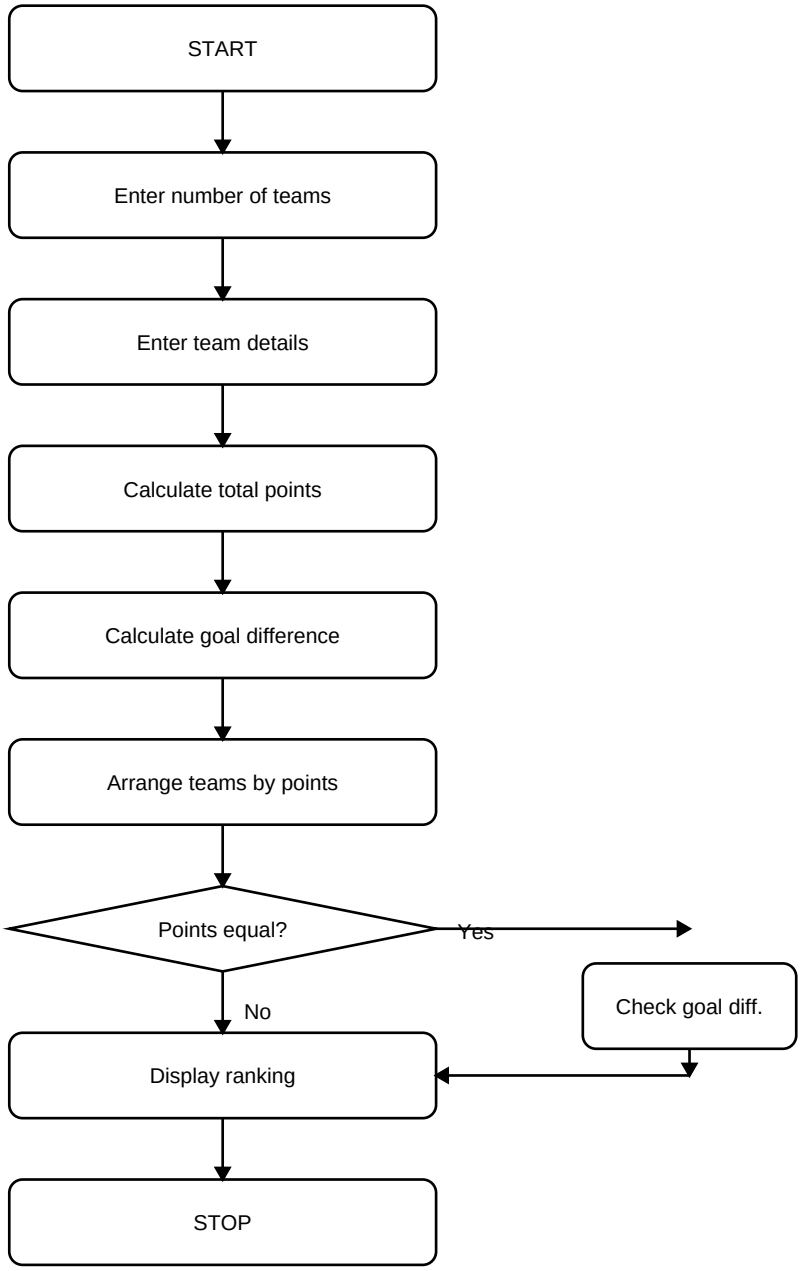
• Arrange teams from highest points to lowest points.

• If points are equal, compare goal difference.

• Display final ranking.

• Stop.

5. Flowchart


START
Enter number of teams
Enter team details
Calculate total points
Calculate goal difference
Arrange teams by points
Points equal?
Yes
No
Check goal diff.
Display ranking
STOP

6. Pseudocode

```
START
INPUT number of teams
CREATE empty team list

REPEAT for every team
    INPUT team name
    INPUT wins and draws
    INPUT goals scored and goals conceded
    points = wins * 2 + draws
    goal difference = goals scored - goals conceded
    STORE all values
END REPEAT

ARRANGE teams by highest points
IF points are equal
    CHECK goal difference
END IF

DISPLAY final ranking
STOP
```

7. Python Source Code

```
teams = []

n = int(input("Enter number of teams: "))

for i in range(n):
    print("Enter team details")
    name = input("Enter team name: ")
    wins = int(input("Enter wins: "))
    draws = int(input("Enter draws: "))
    goals_scored = int(input("Enter goals scored: "))
    goals_conceded = int(input("Enter goals conceded: "))

    points = wins * 2 + draws
    goal_difference = goals_scored - goals_conceded
    teams.append([name, points, goal_difference])

for i in range(n):
    for j in range(i + 1, n):
        if teams[i][1] < teams[j][1]:
            temp = teams[i]
            teams[i] = teams[j]
            teams[j] = temp
        elif teams[i][1] == teams[j][1]:
            if teams[i][2] < teams[j][2]:
                temp = teams[i]
                teams[i] = teams[j]
                teams[j] = temp

print("FINAL RANKING")
print("Rank Team Points Goal Difference")
for i in range(n):
    print(i + 1, teams[i][0], teams[i][1], teams[i][2])
```

8. Test Cases

Team	Wins	Draws	Goals Scored	Goals Conceded	Points	Goal Diff.
Lions	4	1	10	5	9	5
Tigers	4	1	8	5	9	3
Eagles	2	2	6	4	6	2
Sharks	1	1	4	7	3	-3

9. Sample Output

```
FINAL RANKING
Rank Team Points Goal Difference
1 Lions 9 5
2 Tigers 9 3
3 Eagles 6 2
4 Sharks 3 -3
```

10. Conclusion

This program stores sports tournament details, calculates team points and goal difference, and displays the final ranking. It uses basic input, lists, loops, if-else statements, arithmetic and simple sorting.

Case Study: Hospital Ward Occupancy Monitoring

1. Problem Statement

A hospital has different wards with a fixed bed capacity. This program stores bed occupancy counts using a nested list, calculates utilization percentages, detects capacity violations, and identifies peak-demand periods.

2. Objectives

• Store ward occupancy counts using a nested list.

• Calculate bed utilization percentage.

• Detect occupancy greater than ward capacity.

• Identify peak demand when all beds are occupied.

3. Assumptions

• The ward capacity is 10 beds.

• Occupancy data is stored in a simple nested list.

• Utilization percentage = (Occupied beds / Capacity) x 100.

• If occupied beds are greater than capacity, it is a capacity violation.

• If occupied beds are equal to capacity, it is peak demand.

4. Algorithm

• Start.

• Store bed occupancy values in a nested list.

• Set capacity to 10.

• Read each occupied-bed value using loops.

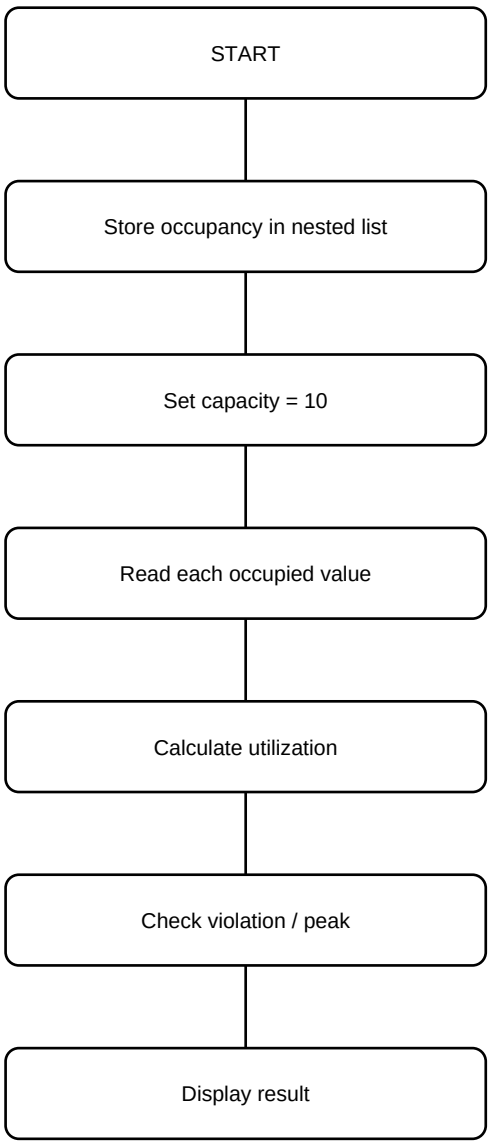
• Calculate utilization percentage.

• If occupied beds are greater than capacity, display Capacity exceeded.

• If occupied beds are equal to capacity, display Peak demand.

• Stop.

5. Flowchart


START
Store occupancy in nested list
Set capacity = 10
Read each occupied value
Calculate utilization
Check violation / peak
Display result

6. Pseudocode

```
START
beds = [[8, 9, 10], [6, 7, 8]]
capacity = 10

FOR each ward in beds
    FOR each occupied value in ward
        percentage = (occupied / capacity) * 100
        DISPLAY occupied and percentage

        IF occupied > capacity
            DISPLAY "Capacity exceeded"
        END IF

        IF occupied == capacity
            DISPLAY "Peak demand"
        END IF
    END FOR
END FOR
STOP
```

7. Python Source Code

```
beds = [
    [8, 9, 10],
    [6, 7, 8]
]

capacity = 10

for ward in beds:
    for occupied in ward:
        percentage = (occupied / capacity) * 100

        print("Occupied beds:", occupied)
        print("Utilization:", percentage)

        if occupied > capacity:
            print("Capacity exceeded")

        if occupied == capacity:
            print("Peak demand")
```

8. Test Cases

Ward	Day 1	Day 2	Day 3	Capacity
Ward 1	8	9	10	10
Ward 2	6	7	8	10

9. Sample Output

Sample outputs are shown in separate boxes for easy understanding.

Output Box 1 - Normal Occupancy

```
Occupied beds: 8

Utilization: 80.0
```

Output Box 2 - Peak Demand

```
Occupied beds: 10
```

```
Utilization: 100.0

Peak demand
```

Output Box 3 - Another Normal Value

```
Occupied beds: 7

Utilization: 70.0
```

10. Conclusion

This simple program uses a nested list, basic loops, one percentage formula, and simple if conditions to monitor hospital ward occupancy. It calculates utilization, checks capacity violations, and identifies peak demand.

Case Study: Student Email and Identifier Validation

1. Problem Statement

Students enter an email-like string and a registration identifier. This program checks them using simple formatting rules and displays whether each value is valid or invalid.

2. Objectives

• Check whether the email contains @ and a dot.

• Check whether the registration ID has the correct length.

• Check whether the ID contains the required letters and digits.

• Display Valid or Invalid clearly.

3. Assumptions / Formatting Rules

• Email must contain @ and a dot.

• Registration ID must contain 2 letters followed by 6 digits.

• Example valid ID: CS123456.

• Spaces are not used in the registration ID.

4. Algorithm

• Start.

• Enter student email.

• Enter registration ID.

• Check whether @ and dot are present in the email.

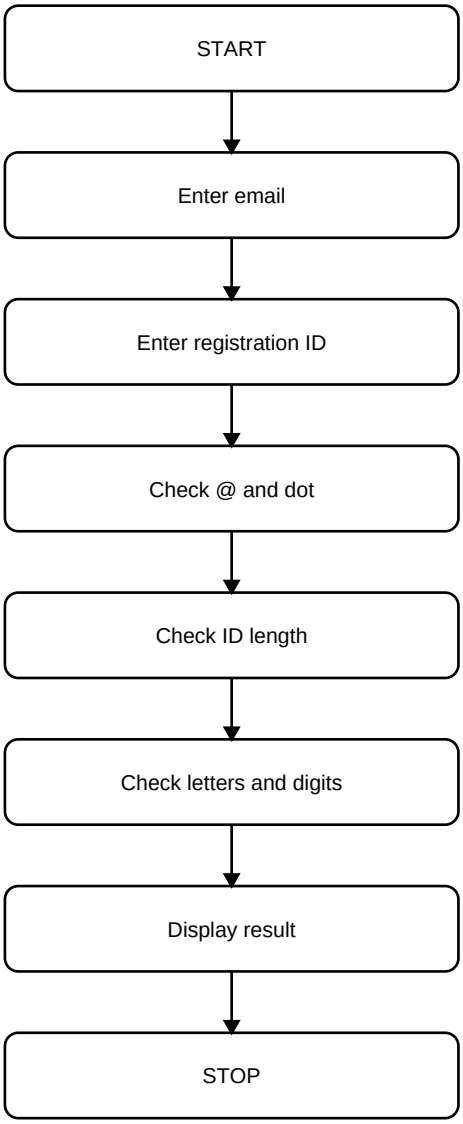
• Check whether the registration ID length is 8.

• Check first 2 characters are letters and remaining 6 characters are digits.

• Display email result and registration ID result.

• Stop.

5. Flowchart


START
Enter email
Enter registration ID
Check @ and dot
Check ID length
Check letters and digits
Display result
STOP

6. Pseudocode

```
START
INPUT email
INPUT registration ID

IF @ is in email AND dot is in email
    DISPLAY "Email is valid"
ELSE
    DISPLAY "Email is invalid"
END IF

IF ID length is 8 AND first 2 are letters AND remaining are digits
    DISPLAY "Registration ID is valid"
ELSE
    DISPLAY "Registration ID is invalid"
END IF
STOP
```

7. Python Source Code

```
email = input("Enter student email: ")
reg = input("Enter registration ID: ")

if "@" in email and "." in email:
    print("Email is valid")
else:
    print("Email is invalid")

if len(reg) == 8 and reg[0:2].isalpha() and reg[2:].isdigit():
    print("Registration ID is valid")
else:
    print("Registration ID is invalid")
```

8. Test Cases

Email	Registration ID	Email Result	ID Result
jaya@college.edu	CS123456	Valid	Valid
jayacollege.edu	CS123456	Invalid	Valid
jaya@college.edu	C1234567	Valid	Invalid
jaya@college.edu	CS12AB56	Valid	Invalid

9. Sample Output

Sample outputs are shown in separate boxes for easy understanding.

Sample Output 1 - Both Valid

```
Enter student email: jaya@college.edu

Enter registration ID: CS123456

Email is valid

Registration ID is valid
```

Sample Output 2 - Invalid Email

```
Enter student email: jayacollege.edu

Enter registration ID: CS123456

Email is invalid

Registration ID is valid
```

Sample Output 3 - Invalid Registration ID

```
Enter student email: jaya@college.edu

Enter registration ID: CS12AB56

Email is valid

Registration ID is invalid
```

10. Conclusion

This basic program checks a student email and registration identifier using simple conditions. It uses input, if-else statements, string checking, isalpha(), isdigit(), and length checking, so it is easy to understand.

Case Study: Integrated Campus Energy and Water Audit

1. Problem Statement

A college campus has different buildings that use electricity and water every day. This program stores energy and water consumption, calculates costs and percentage changes, identifies excessive consumption, calculates a simple audit score, and ranks the buildings.

2. Objectives

• Store energy and water consumption of campus buildings.

• Calculate energy cost, water cost, and total cost.

• Calculate percentage change in consumption.

• Identify buildings with excessive consumption.

• Calculate an audit score and rank the buildings.

3. Assumptions

• Energy cost = Energy units x 5.

• Water cost = Water units x 2.

• Percentage change = ((Current use - Previous use) / Previous use) x 100.

• Energy use above 500 units or water use above 300 units is considered excessive.

• Audit score starts from 10.

• Subtract 2 for excessive energy use and subtract 2 for excessive water use.

• Higher audit score gives a better rank.

4. Algorithm

• Start.

• Enter number of buildings.

• Enter building name, previous energy use, current energy use, and current water use.

• Calculate energy cost and water cost.

• Calculate total cost.

• Calculate percentage change in energy consumption.

• Check excessive energy and water consumption.

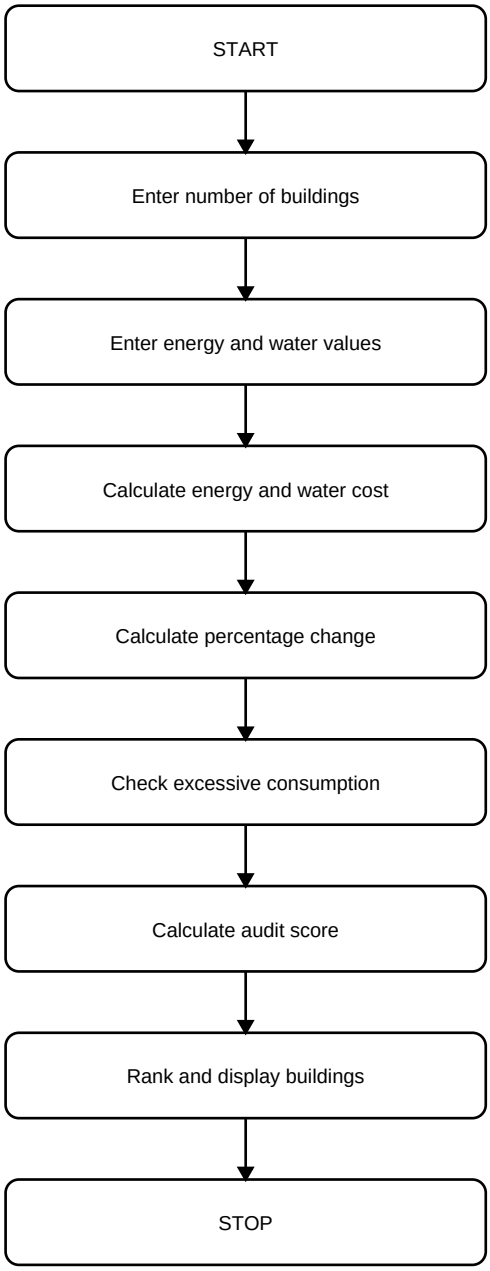
• Calculate audit score.

• Arrange buildings from highest audit score to lowest audit score.

• Display final ranking.

• Stop.

5. Flowchart


START
Enter number of buildings
Enter energy and water values
Calculate energy and water cost
Calculate percentage change
Check excessive consumption
Calculate audit score
Rank and display buildings
STOP

6. Pseudocode

```
START
INPUT number of buildings
CREATE empty building list
REPEAT for every building
    INPUT building name
    INPUT previous energy use
    INPUT current energy use
    INPUT current water use
    energy cost = current energy use * 5
    water cost = current water use * 2
    total cost = energy cost + water cost
    percentage change = ((current energy use - previous energy use) / previous energy use) * 100
    score = 10
    IF current energy use > 500
        score = score - 2
    END IF
    IF current water use > 300
        score = score - 2
    END IF
    STORE building details
END REPEAT
ARRANGE buildings by highest score
DISPLAY final ranking
STOP
```

7. Python Source Code

```
buildings = []
n = int(input("Enter number of buildings: "))
for i in range(n):
    print("Enter building details")
    name = input("Enter building name: ")
    previous_energy = float(input("Enter previous energy use: "))
    current_energy = float(input("Enter current energy use: "))
    current_water = float(input("Enter current water use: "))
    energy_cost = current_energy * 5
    water_cost = current_water * 2
    total_cost = energy_cost + water_cost
    change = ((current_energy - previous_energy) / previous_energy) * 100
    score = 10
    if current_energy > 500:
        score = score - 2
    if current_water > 300:
        score = score - 2
    buildings.append([name, current_energy, current_water, total_cost, change, score])
for i in range(n):
    for j in range(i + 1, n):
        if buildings[i][5] < buildings[j][5]:
            temp = buildings[i]
            buildings[i] = buildings[j]
            buildings[j] = temp
print("CAMPUS AUDIT RANKING")
print("Rank Building Energy Water Cost Change Score")
for i in range(n):
    print(i + 1, buildings[i][0], buildings[i][1], buildings[i][2], buildings[i][3], round(buildi
ngs[i][4], 2), buildings[i][5])
```

8. Test Cases

Building	Prev. Energy	Current Energy	Water	Total Cost	Change %	Score
Library	400	450	200	2650	12.50	10
Block A	450	550	250	3250	22.22	8
Block B	500	600	350	3700	20.00	6
Lab	420	480	320	3040	14.29	8

9. Sample Output

Sample output is separated into clear boxes for easy understanding.

Input Box

```
Number of buildings: 4

Library: Previous Energy 400, Current Energy 450, Water 200

Block A: Previous Energy 450, Current Energy 550, Water 250

Block B: Previous Energy 500, Current Energy 600, Water 350

Lab: Previous Energy 420, Current Energy 480, Water 320
```

Calculated Values Box

```
Library: Cost 2650, Change 12.50%, Score 10

Block A: Cost 3250, Change 22.22%, Score 8

Block B: Cost 3700, Change 20.00%, Score 6

Lab: Cost 3040, Change 14.29%, Score 8
```

Final Ranking Box

```
Rank 1 - Library - Score 10

Rank 2 - Block A - Score 8

Rank 3 - Lab - Score 8

Rank 4 - Block B - Score 6
```

10. Conclusion

This program analyses basic campus energy and water usage, calculates costs and percentage changes, identifies excessive consumption, gives each building an audit score, and displays the final ranking. It uses simple input, lists, loops, if statements, arithmetic, and basic sorting.

Case Study: Student Timetable Conflict Detection

1. Problem Statement

A student timetable contains courses, days, periods, and rooms. This program compares two course schedules represented using simple strings and tuples, detects overlapping periods, identifies an unavailable room, and validates the proposed timetable.

2. Objectives

• Store course details using strings and tuples.

• Compare two course schedules.

• Detect courses having the same day and period.

• Identify an unavailable room.

• Display whether the proposed timetable is valid or invalid.

3. Assumptions

• Each schedule is stored as a tuple: (course, day, period, room).

• A conflict occurs when two courses have the same day and the same period.

• Room R5 is considered unavailable for this basic example.

• If there is no time conflict and the rooms are available, the timetable is valid.

4. Algorithm

• Start.

• Enter details of Course 1 and Course 2.

• Store each course schedule in a tuple.

• Compare the day and period of both courses.

• If both day and period are equal, report a timetable conflict.

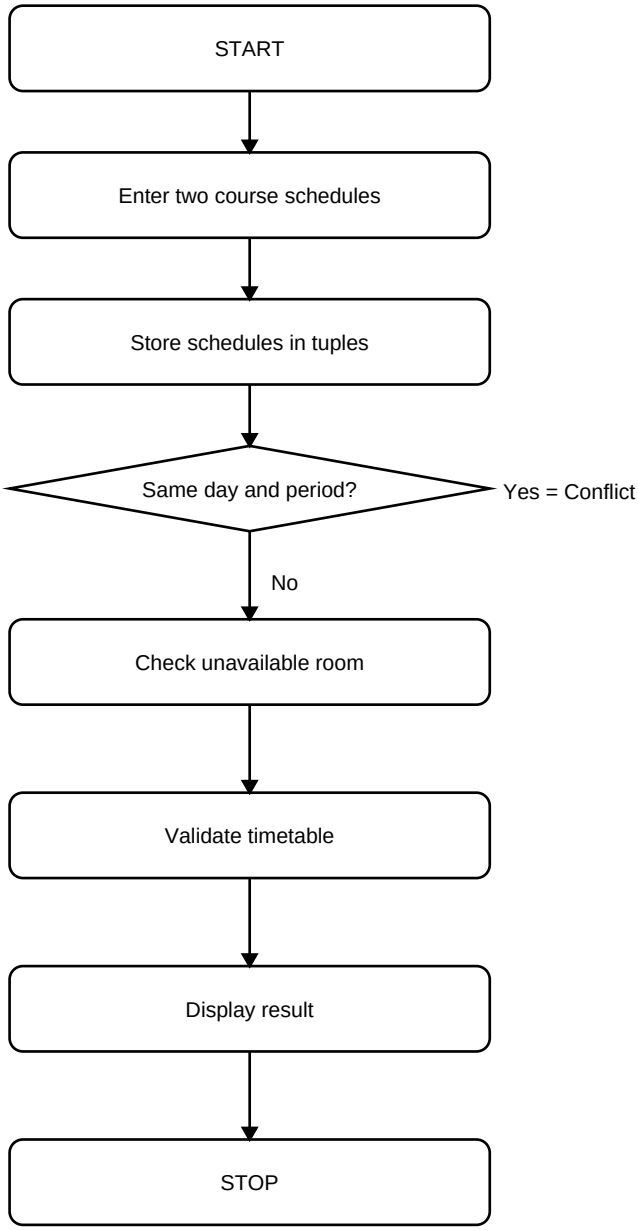
• Check whether either course uses unavailable room R5.

• If there is no conflict and no unavailable room, display Valid Timetable.

• Otherwise display Invalid Timetable.

• Stop.

5. Flowchart


START
Enter two course schedules
Store schedules in tuples
Same day and period?
Yes = Conflict
No
Check unavailable room
Validate timetable
Display result
STOP

6. Pseudocode

```
START
INPUT course1, day1, period1, room1
INPUT course2, day2, period2, room2
schedule1 = (course1, day1, period1, room1)
schedule2 = (course2, day2, period2, room2)
conflict = False
IF day1 == day2 AND period1 == period2
    DISPLAY "Time Conflict"
    conflict = True
END IF
IF room1 == "R5" OR room2 == "R5"
    DISPLAY "Room Unavailable"
    conflict = True
END IF
IF conflict == False
    DISPLAY "Valid Timetable"
ELSE
    DISPLAY "Invalid Timetable"
END IF
STOP
```

7. Python Source Code

```
course1 = input("Enter first course: ")
day1 = input("Enter day: ")
period1 = input("Enter period: ")
room1 = input("Enter room: ")

course2 = input("Enter second course: ")
day2 = input("Enter day: ")
period2 = input("Enter period: ")
room2 = input("Enter room: ")

schedule1 = (course1, day1, period1, room1)
schedule2 = (course2, day2, period2, room2)

conflict = False

if day1 == day2 and period1 == period2:
    print("Time Conflict")
    conflict = True

if room1 == "R5" or room2 == "R5":
    print("Room Unavailable")
    conflict = True

if conflict == False:
    print("Valid Timetable")
else:
    print("Invalid Timetable")
```

8. Test Cases

Course 1	Day/Period	Room	Course 2	Day/Period	Room	Result
DBMS	Mon / P1	R1	Java	Tue / P2	R2	Valid
DBMS	Mon / P1	R1	Java	Mon / P1	R2	Time Conflict
Python	Wed / P2	R5	SE	Thu / P1	R2	Room Unavailable
AIML	Fri / P2	R2	DM	Fri / P1	R4	Valid

9. Sample Output

Sample outputs are shown in separate boxes for easy understanding.

Output Box 1 - Valid Timetable

```
DBMS: Mon, P1, R1

Java: Tue, P2, R2

Valid Timetable
```

Output Box 2 - Time Conflict

```
DBMS: Mon, P1, R1

Java: Mon, P1, R2

Time Conflict

Invalid Timetable
```

Output Box 3 - Room Unavailable

```
Python: Wed, P2, R5

SE: Thu, P1, R2

Room Unavailable

Invalid Timetable
```

10. Conclusion

This basic program compares two student course schedules, detects an overlapping day and period, checks an unavailable room, and validates the proposed timetable. It uses simple input, strings, tuples, if statements, and Boolean values.

CASE STUDY 1: SMART PUBLIC TRANSPORT FARE RECONCILIATION

1. Problem Statement

Public transport buses carry different categories of passengers, such as general passengers and students. Each category may have a different fare. This program calculates the expected revenue for each bus route using the fare and passenger counts. It compares the expected revenue with the actual collected revenue and identifies a shortage, excess collection, or no discrepancy.

2. Objectives

1. To calculate the fare revenue from general passengers and student passengers.
2. To calculate the expected revenue for each bus route.
3. To compare expected revenue with collected revenue.
4. To identify shortages, excess collections, or no discrepancy.
5. To calculate total expected and collected revenue for all routes.
6. To display the overall revenue difference clearly.

3. Assumptions

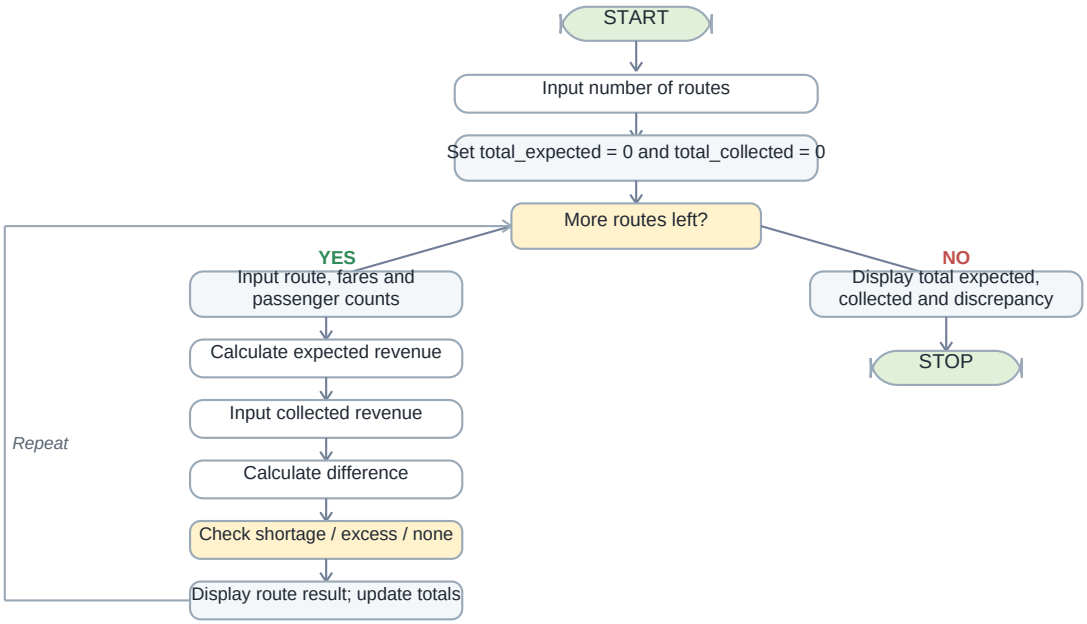
1. The number of routes is greater than zero.
2. Each route has a general fare and a student fare.
3. Passenger counts are whole numbers and cannot be negative.
4. The fare amounts entered by the user are correct.
5. The collected revenue is entered by the user.
6. All amounts are in Indian rupees (■).
7. Difference = collected revenue – expected revenue.
8. A negative difference means shortage; a positive difference means excess collection.

4. Algorithm

1. Start the program.
2. Enter the number of bus routes.
3. Initialize total expected revenue and total collected revenue to zero.
4. For each route, enter the route name, general fare, student fare, and passenger counts.
5. Calculate expected revenue = (general passengers × general fare) + (student passengers × student fare).
6. Enter the collected revenue for the route.
7. Calculate difference = collected revenue – expected revenue.
8. If the difference is negative, display the shortage.
9. If the difference is positive, display the excess collection.
10. Otherwise, display no discrepancy.
11. Add the route's expected and collected revenues to the totals.
12. Repeat until all routes have been processed.
13. Display total expected revenue, total collected revenue, and total discrepancy.
14. Stop the program.

5. Flowchart

The flowchart shows the main steps followed by the program.



6. Pseudocode

```
START
INPUT number_of_routes
SET total_expected = 0
SET total_collected = 0

FOR each route:
    INPUT route_name
    INPUT general_fare, student_fare
    INPUT general_passengers, student_passengers
    expected = (general_passengers * general_fare) +
               (student_passengers * student_fare)
    INPUT collected_revenue
    difference = collected_revenue - expected

    IF difference < 0:
        DISPLAY shortage = -difference
    ELSE IF difference > 0:
        DISPLAY excess = difference
    ELSE:
        DISPLAY "No discrepancy"

    total_expected = total_expected + expected
    total_collected = total_collected + collected_revenue
END FOR

DISPLAY total_expected
DISPLAY total_collected
DISPLAY total_collected - total_expected
STOP
```

7. Python Source Code

```
routes = int(input("Enter number of routes: "))

totalexpected = 0
totalcollected = 0

for i in range(routes):
    print("Route", i + 1)
    name = input("Enter route name: ")
    fare = float(input("Enter general fare: "))
    studentfare = float(input("Enter student fare: "))
    general = int(input("Enter general passengers: "))
    students = int(input("Enter student passengers: "))

    expected = general * fare + students * studentfare
    collected = float(input("Enter collected revenue: "))
    difference = collected - expected

    print("Route:", name)
    print("Expected revenue:", expected)
    print("Collected revenue:", collected)

    if difference < 0:
        print("Shortage:", -difference)
    elif difference > 0:
        print("Excess:", difference)
    else:
        print("No discrepancy")

    totalexpected = totalexpected + expected
    totalcollected = totalcollected + collected

print("Total expected revenue:", totalexpected)
print("Total collected revenue:", totalcollected)
print("Total discrepancy:", totalcollected - totalexpected)
```

8. Test Cases

The following test cases use the same values as the sample run. Expected revenue is calculated from the fare multiplied by the passenger count in each category.

Test case	Route	Expected revenue	Collected revenue	Expected result
1	Gannavaram	■2,939	■67	Shortage: ■2,872
2	Vijayawada	■2,568	■48	Shortage: ■2,520
Total	Both routes	■5,507	■115	Shortage: ■5,392

Calculation Check

Route 1 – Gannavaram: General revenue = 45 × ■23 = ■1,035. Student revenue = 56 × ■34 = ■1,904. Expected revenue = ■2,939. Shortage = ■2,939 – ■67 = ■2,872.

Route 2 – Vijayawada: General revenue = 37 × ■24 = ■888. Student revenue = 48 × ■35 = ■1,680. Expected revenue = ■2,568. Shortage = ■2,568 – ■48 = ■2,520.

Overall totals: Total expected revenue = ■5,507. Total collected revenue = ■115. Total discrepancy = ■115 – ■5,507 = –■5,392, which means an overall shortage of ■5,392.

9. Sample Output

Sample run for two routes:

```
Enter number of routes: 2
Route 1
Enter route name: GANNAVARAM
Enter general fare: 23
Enter student fare: 34
Enter general passengers: 45
Enter student passengers: 56
Enter collected revenue: 67
Route: GANNAVARAM
Expected revenue: 2939.0
Collected revenue: 67.0
Shortage: 2872.0
```

```
Route 2
Enter route name: VIJAYAWADA
Enter general fare: 24
Enter student fare: 35
Enter general passengers: 37
Enter student passengers: 48
Enter collected revenue: 48
Route: VIJAYAWADA
Expected revenue: 2568.0
Collected revenue: 48.0
Shortage: 2520.0

Total expected revenue: 5507.0
Total collected revenue: 115.0
Total discrepancy: -5392.0
```

10. Conclusion

This case study helped me understand how Python can be used to calculate transport fares and check revenue. The program calculates expected revenue for each route, compares it with the collected amount, and identifies shortages, excess collections, or no discrepancy. It also adds the revenues from all routes and displays the final totals. This program uses basic Python concepts such as variables, arithmetic operators, loops, input/output statements, and conditional statements.

Note: The program assumes that the fare and passenger information entered by the user is correct. In a larger real-world system, input validation and secure digital ticket records could be added.

Case Study: Hospital Resource Allocation and Capacity Monitoring

1. Problem Statement

Hospitals have different wards, such as Emergency, General, ICU, and Maternity. Each ward has a limited number of beds, staff members, and medical equipment. This program checks the resources in each ward, identifies shortages, and assigns a priority level using simple rules.

2. Objectives

• To store ward details using nested lists.

• To calculate available beds in each ward.

• To check whether staff and equipment are enough.

• To identify resource shortages.

• To assign HIGH, MEDIUM, or LOW priority.

3. Assumptions

• Each ward list contains: name, total beds, occupied beds, available staff, required staff, available equipment, and required equipment.

• Available beds = total beds – occupied beds.

• A bed shortage is reported when fewer than 2 beds are available.

• Staff or equipment shortage occurs when available amount is less than the required amount.

• Two or more types of shortage = HIGH priority; one = MEDIUM; none = LOW.

• This is an educational example, not for real hospital decisions.

4. Algorithm

1. Start the program.

2. Create a nested list with details of each ward.

3. Take one ward from the list.

4. Calculate available beds: total beds – occupied beds.

5. Check beds, staff, and equipment for shortages.

6. Store the types of shortages found.

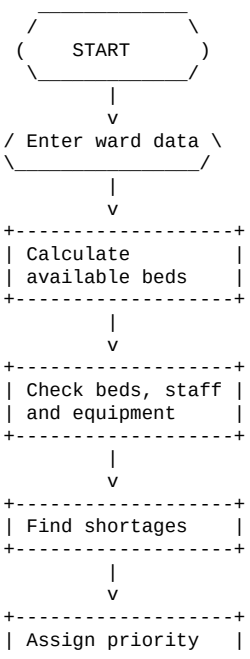
7. Assign HIGH, MEDIUM, or LOW priority.

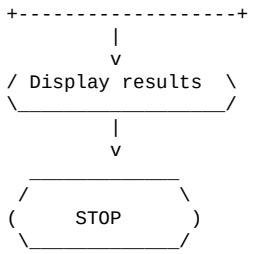
8. Display ward details, shortages, and priority.

9. Repeat for all wards.

10. Stop the program.

5. Flowchart (Sketch)





Sketch symbols: oval for Start/Stop, parallelogram for Input/Output, and rectangle for processing.

6. Pseudocode

```
BEGIN
    wards = list of ward details
    FOR each ward in wards
        available_beds = total_beds - occupied_beds
        shortages = empty list

        IF available_beds < 2
            add "Beds" to shortages
        IF available_staff < required_staff
            add "Staff" to shortages
        IF available_equipment < required_equipment
            add "Equipment" to shortages

        IF number of shortages >= 2
            priority = "HIGH"
        ELSE IF number of shortages == 1
            priority = "MEDIUM"
        ELSE
            priority = "LOW"

        DISPLAY ward details, shortages, priority
    END FOR
END
```

7. Python Source Code

```
# Hospital Resource Allocation and Capacity Monitoring
# Ward: [name, total beds, occupied beds, staff,
#        required staff, equipment, required equipment]

wards = [
    ["Emergency", 20, 18, 4, 6, 2, 4],
    ["General", 30, 20, 8, 7, 5, 4],
    ["ICU", 10, 10, 3, 5, 2, 3],
    ["Maternity", 15, 10, 4, 4, 3, 3]
]

for ward in wards:
    name = ward[0]
    total_beds = ward[1]
    occupied_beds = ward[2]
    staff = ward[3]
    required_staff = ward[4]
    equipment = ward[5]
    required_equipment = ward[6]

    available_beds = total_beds - occupied_beds
    shortages = []

    if available_beds < 2:
        shortages.append("Beds")
    if staff < required_staff:
        shortages.append("Staff")
    if equipment < required_equipment:
        shortages.append("Equipment")

    if len(shortages) >= 2:
        priority = "HIGH"
    elif len(shortages) == 1:
        priority = "MEDIUM"
    else:
        priority = "LOW"

    print("\\nWard:", name)
    print("Available beds:", available_beds)
    print("Shortages:", ", ".join(shortages) if shortages else "None")
    print("Priority:", priority)
```

8. Test Cases

Ward	Available beds	Shortages	Expected priority
Emergency	2	Staff, Equipment	HIGH

Ward	Available beds	Shortages	Expected priority
General	10	None	LOW
ICU	0	Beds, Staff, Equipment	HIGH
Maternity	5	None	LOW

9. Sample Output

```
Ward: Emergency
Available beds: 2
Shortages: Staff, Equipment
Priority: HIGH

Ward: General
Available beds: 10
Shortages: None
Priority: LOW

Ward: ICU
Available beds: 0
Shortages: Beds, Staff, Equipment
Priority: HIGH

Ward: Maternity
Available beds: 5
Shortages: None
Priority: LOW
```

10. Conclusion

This program uses nested lists, loops, and if statements to check hospital resources. It calculates available beds, finds shortages in staff and equipment, and assigns a priority level. It is a simple example suitable for a first-year B.Tech student.

Case Study: Examination Result and Grace-Mark Validation

1. Problem Statement

Students write examinations in multiple subjects. Each subject has its own maximum marks and pass marks. This program checks the marks, gives grace marks according to a simple policy when allowed, makes sure final marks do not exceed maximum marks, and displays the final result.

2. Objectives

• To store marks of different subjects using nested lists.

• To check pass marks subject by subject.

• To apply grace marks only when allowed.

• To keep final marks within the maximum marks.

• To display subject results and the overall result.

3. Assumptions

• Each subject contains its name, obtained marks, maximum marks, and pass marks.

• A maximum of 5 grace marks can be given in total.

• Grace marks can be used for only one subject, if the student is short by 5 marks or less.

• Only the marks needed to reach the pass mark are added.

• Final marks cannot be greater than maximum marks.

• Overall result is PASS only if all subjects are passed. This is a sample rule for learning.

4. Algorithm

1. Start.

2. Enter subject details and marks.

3. Set grace limit to 5 marks.

4. Check the marks of each subject.

5. If a student is short by 5 marks or less and grace marks have not been used, add only the marks needed to pass.

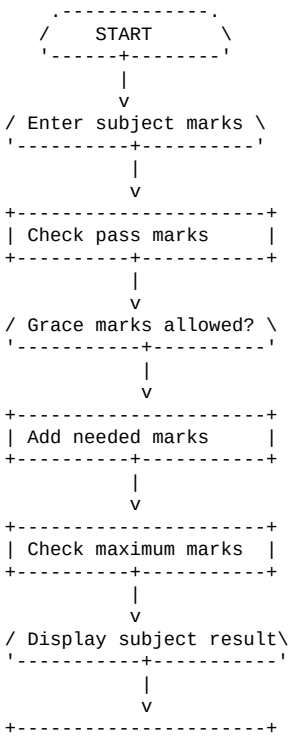
6. Make sure final marks do not exceed maximum marks.

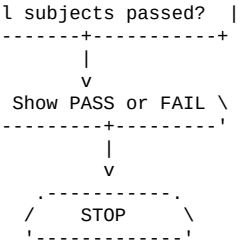
7. Display original marks, final marks, and result.

8. Display overall PASS if all subjects are passed; otherwise display FAIL.

9. Stop.

5. Flowchart (Sketch)





Draw Start/Stop as ovals, input/output as parallelograms, processing as rectangles, and questions as diamonds in your notebook.

6. Pseudocode

```
BEGIN
    grace_used = 0
    all_passed = TRUE

    FOR each subject
        final_marks = obtained_marks

        IF obtained_marks < pass_marks
            needed = pass_marks - obtained_marks
            IF needed <= 5 AND grace_used == 0
                final_marks = final_marks + needed
                grace_used = needed
            END IF
        END IF

        IF final_marks > maximum_marks
            final_marks = maximum_marks
        END IF

        IF final_marks < pass_marks
            result = "FAIL"
            all_passed = FALSE
        ELSE
            result = "PASS"
        END IF

        DISPLAY subject, final_marks, result
    END FOR

    IF all_passed
        DISPLAY "OVERALL PASS"
    ELSE
        DISPLAY "OVERALL FAIL"
    END IF
END
```

7. Python Source Code

```
subjects = [
    ["English", 68, 100, 40],
    ["Mathematics", 37, 100, 40],
    ["Programming", 75, 100, 40],
    ["Physics", 32, 100, 40]
]

grace = 5
used = 0
all_pass = True

for sub in subjects:
    name = sub[0]
    marks = sub[1]
    maximum = sub[2]
    pass_mark = sub[3]
    final = marks

    if marks < pass_mark:
        need = pass_mark - marks
        if need <= grace and used == 0:
            final = marks + need
            used = need

    if final > maximum:
        final = maximum

    if final >= pass_mark:
        result = "PASS"
    else:
        result = "FAIL"
        all_pass = False

    print("Subject:", name)
    print("Marks:", marks)
    print("Final Marks:", final)
    print("Result:", result)
```

```
    print("--------------------")

if all_pass:
    print("Overall Result: PASS")
else:
    print("Overall Result: FAIL")
```

8. Test Cases

Subject	Marks	Final marks	Result
English	68	68	PASS
Mathematics	37	40 (3 grace)	PASS
Programming	75	75	PASS
Physics	32	32	FAIL

9. Sample Output

```
Subject: English
Marks: 68
Final Marks: 68
Result: PASS
--------------------
Subject: Mathematics
Marks: 37
Final Marks: 40
Result: PASS
--------------------
Subject: Programming
Marks: 75
Final Marks: 75
Result: PASS
--------------------
Subject: Physics
Marks: 32
Final Marks: 32
Result: FAIL
--------------------
Overall Result: FAIL
```

10. Conclusion

This program checks marks for multiple subjects, applies grace marks according to the given rule, and displays subject-wise and overall results. It uses lists, a for loop, and if statements, making it suitable for a first-year B.Tech student.

Case Study: Student Feedback Sentiment Scoring

1. Problem Statement

Students give feedback about classes and college facilities. This program checks feedback using positive and negative keywords, calculates a score, and displays whether the feedback is Positive, Negative, or Neutral.

2. Objectives

• Count positive and negative keywords in feedback.

• Calculate a score.

• Display the result.

3. Assumptions

• Positive keywords increase the positive count by 1; negative keywords increase the negative count by 1.

• Score = positive count – negative count.

• Score above 0 is Positive, below 0 is Negative, and 0 is Neutral.

• Enter feedback in lowercase.

4. Algorithm

• Start.

• Enter a feedback sentence.

• Set positive and negative counts to zero.

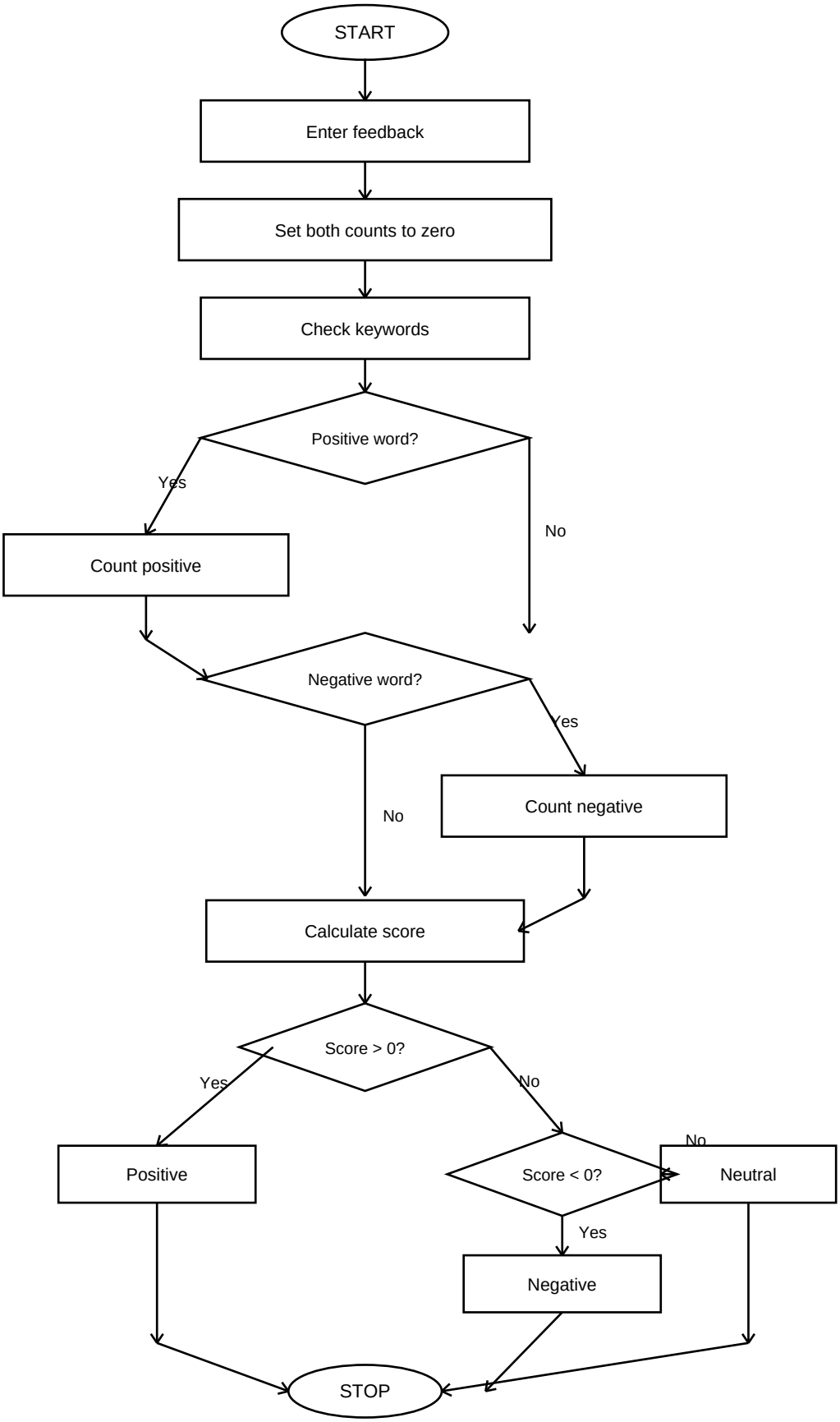
• Check the sentence for positive and negative keywords.

• Calculate score = positive count – negative count.

• Display Positive, Negative, or Neutral based on the score.

• Stop.

5. Flowchart


START
Enter feedback
Set both counts to zero
Check keywords
Positive word?
Yes
No
Count positive
Negative word?
Yes
No
Count negative
Calculate score
Score > 0?
Yes
No
Positive
Score < 0?
No
Neutral
Yes
Negative
STOP

6. Pseudocode

```
START
INPUT feedback
positive_count = 0
negative_count = 0

IF "good" is in feedback
    positive_count = positive_count + 1
IF "helpful" is in feedback
    positive_count = positive_count + 1
IF "excellent" is in feedback
    positive_count = positive_count + 1
IF "clear" is in feedback
    positive_count = positive_count + 1
IF "interesting" is in feedback
    positive_count = positive_count + 1

IF "bad" is in feedback
    negative_count = negative_count + 1
IF "boring" is in feedback
    negative_count = negative_count + 1
IF "poor" is in feedback
    negative_count = negative_count + 1
IF "difficult" is in feedback
    negative_count = negative_count + 1
IF "confusing" is in feedback
    negative_count = negative_count + 1

score = positive_count - negative_count
IF score > 0, DISPLAY "Positive"
ELSE IF score < 0, DISPLAY "Negative"
ELSE, DISPLAY "Neutral"
STOP
```

7. Python Source Code

```
feedback = input("Enter feedback in lowercase: ")

positive_count = 0
negative_count = 0

if "good" in feedback:
    positive_count = positive_count + 1
if "helpful" in feedback:
    positive_count = positive_count + 1
if "excellent" in feedback:
    positive_count = positive_count + 1
if "clear" in feedback:
    positive_count = positive_count + 1
if "interesting" in feedback:
    positive_count = positive_count + 1

if "bad" in feedback:
    negative_count = negative_count + 1
if "boring" in feedback:
    negative_count = negative_count + 1
if "poor" in feedback:
    negative_count = negative_count + 1
if "difficult" in feedback:
    negative_count = negative_count + 1
if "confusing" in feedback:
    negative_count = negative_count + 1

score = positive_count - negative_count

if score > 0:
    result = "Positive"
elif score < 0:
    result = "Negative"
else:
    result = "Neutral"

print("Positive count:", positive_count)
print("Negative count:", negative_count)
print("Score:", score)
```

```
print("Result:", result)
```

8. Test Cases

Input feedback	Positive	Negative	Score	Result
teacher is good and helpful	2	0	2	Positive
lesson is boring and confusing	0	2	-2	Negative
class is clear and interesting	2	0	2	Positive
teaching is good but notes are poor	1	1	0	Neutral
class was okay	0	0	0	Neutral

9. Sample Output

```
Enter feedback in lowercase: teacher is good and helpful
Positive count: 2
Negative count: 0
Score: 2
Result: Positive
```

10. Conclusion

This program counts positive and negative keywords in student feedback, calculates a score, and displays Positive, Negative, or Neutral. It uses input, if-else statements, and basic arithmetic suitable for a first-year engineering student.

Case Study: Food Waste Monitoring System

1. Problem Statement

A college canteen prepares and serves meals every day. Some meals may remain as leftovers. This program compares prepared meals, meals served, and leftovers for several days, calculates the food wastage percentage, and identifies days that exceed the acceptable waste limit.

2. Objectives

• Store prepared meals and meals served for several days.

• Calculate leftover meals.

• Calculate food wastage percentage.

• Identify days with excessive food waste.

• Display the result for every day.

3. Assumptions

• Prepared meals and served meals are stored in simple lists.

• Leftovers = Prepared meals - Meals served.

• Wastage Percentage = (Leftovers / Prepared meals) x 100.

• The acceptable wastage limit is 10%.

• If wastage is more than 10%, the day is marked as High Waste.

4. Algorithm

• Start.

• Store prepared meals and served meals for several days.

• Calculate leftovers for each day.

• Calculate wastage percentage.

• Compare wastage percentage with 10%.

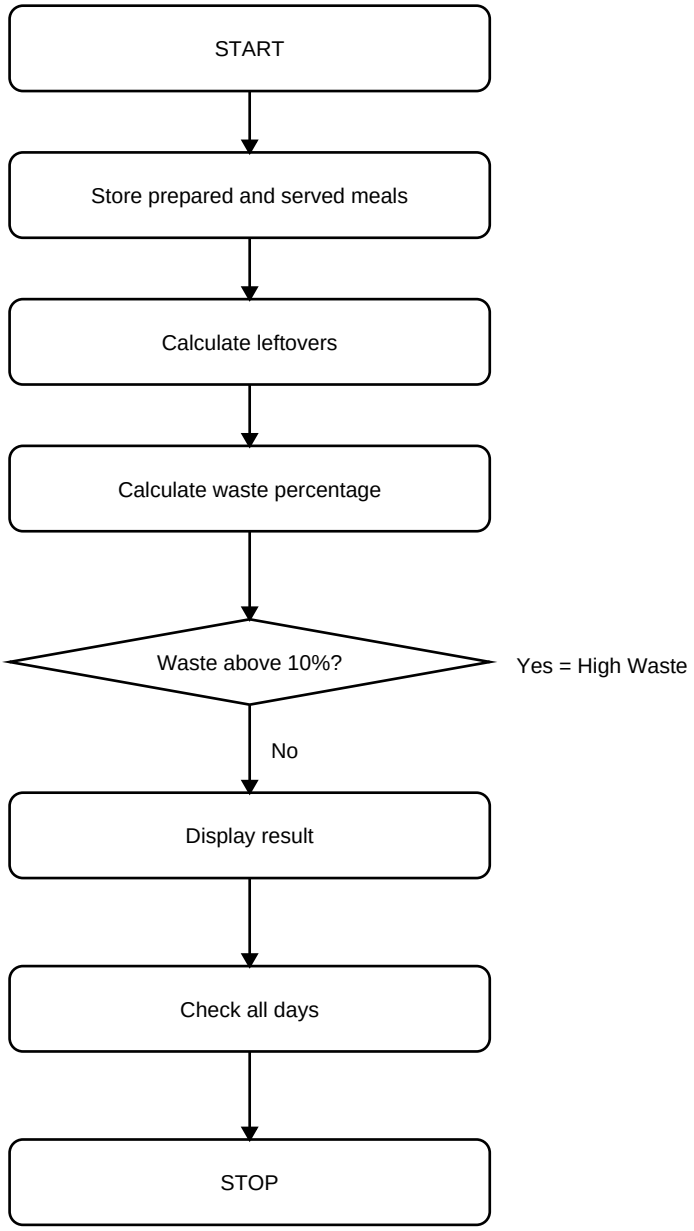
• If wastage is above 10%, display High Waste.

• Otherwise display Acceptable Waste.

• Repeat for all days.

• Stop.

5. Flowchart


START
Store prepared and served meals
Calculate leftovers
Calculate waste percentage
Waste above 10%?
Yes = High Waste
No
Display result
Check all days
STOP

6. Pseudocode

```
START
prepared = [100, 120, 90, 110]
served = [95, 100, 80, 105]

FOR each day
    leftovers = prepared - served
    waste = (leftovers / prepared) * 100
    DISPLAY leftovers
    DISPLAY waste
    IF waste > 10
        DISPLAY "High Waste"
    ELSE
        DISPLAY "Acceptable Waste"
    END IF
END FOR
STOP
```

7. Python Source Code

```
prepared = [100, 120, 90, 110]
served = [95, 100, 80, 105]

for i in range(4):
    leftover = prepared[i] - served[i]
    waste = (leftover / prepared[i]) * 100

    print("Day:", i + 1)
    print("Leftover:", leftover)
    print("Waste:", round(waste, 2))

    if waste > 10:
        print("High Waste")
    else:
        print("Acceptable Waste")
```

8. Test Cases

Day	Prepared	Served	Leftovers	Waste %	Result
Day 1	100	95	5	5.00	Acceptable
Day 2	120	100	20	16.67	High Waste
Day 3	90	80	10	11.11	High Waste
Day 4	110	105	5	4.55	Acceptable

9. Sample Output

Sample outputs are shown in separate boxes for easy understanding.

Output Box 1 - Day 1

```
Leftover: 5

Waste: 5.0

Acceptable Waste
```

Output Box 2 - Day 2

```
Leftover: 20
```

```
Waste: 16.67

High Waste
```

Output Box 3 - Day 3

```
Leftover: 10

Waste: 11.11

High Waste
```

Output Box 4 - Day 4

```
Leftover: 5

Waste: 4.55

Acceptable Waste
```

10. Conclusion

This basic program compares prepared and served meals, calculates leftovers and food wastage percentage, and identifies days with excessive food waste. It uses only simple lists, a loop, arithmetic, and if-else statements.